

Student Name:

Minh Quan Tran

Susheela Narayan Hegde

Team Number:

Group 1

Assignment 2:

Microcode Technical Memo

Contents

Introduction	3
Design Objective and Purpose	3
Overview	4
High-Level Entities (Block Diagram Description)	4
Major Requirements and Specifications	4
Logic Implementation and Status Flags	4
Implementation Method (Design Flow and Tools)	5
Verification and Validation (V&V)	5
Benefits, Assumptions, Risks and Issues	6
Benefits:	6
Assumptions:	6
Risks and Challenges:	6
Conclusion	6
References	7

Introduction

The purpose of this document is to propose the detailed design and implementation plan for a custom Microcode Control Logic Unit as part of a larger Arithmetic Logic Unit (ALU) development project. This proposal outlines the functional architecture, implementation methodology, validation strategy, and projected risks and benefits associated with designing the control logic in-house rather than integrating a pre-built solution.

Design Objective and Purpose

The Control Logic's primary function is to direct the operations of the ALU based on micro-instruction codes. It ensures correct sequencing and operation selection through a series of multiplexers and a Control Address Register (CAR). Within the broader system, this unit acts as the "brain" that coordinates arithmetic, logical, and data flow operations.

Its inclusion is critical for:

- Decoding micro-instructions
- Managing control signals
- Enabling modular and scalable ALU operations

This module significantly contributes to the system's flexibility and maintainability, especially when implementing instruction sets and advanced control features.

Overview

High-Level Entities (Block Diagram Description)

The Microcode Control Logic consists of the following primary components:

- **MUX1:** Selects which ALU operation to activate
- **MUX2:** Selects register operands to feed into ALU inputs

Inputs: Clock, Reset, External Instruction Code, Conditional Flags

Outputs: Control Signals to ALU blocks, Next State Address.

Major Requirements and Specifications

- Clock-synchronized state transitions
- Support for up to 8 ALU operations
- Dynamic operand and operation selection
- Write enable signal for register file

Logic Implementation and Status Flags

In the current control logic implementation, two key multiplexers, MUX1 and MUX2, are used to orchestrate ALU behavior.

MUX1 – Operation Selector:

MUX1 controls which ALU operation is active by enabling a single operation block (such as ADD, SUB, etc.) based on the 4-bit operation code (OP). The operation selection logic uses conditional signal assignment, meaning only one operation is executed at a time. This ensures optimized resource usage and eliminates logic conflicts during computation.

MUX2 – Operand Selector:

MUX2 is responsible for selecting the source operands from a shared register file. It receives control signals SEL_A and SEL_B that specify which registers supply data to the ALU's A and B input buses. This allows for flexible combinations of operands and supports full register-to-register operations.

CAR – Control Address Register (Not Implemented):

Although not currently implemented, the architecture is designed to accommodate a future CAR component. This would allow the system to fetch control signals from a ROM or

microinstruction memory and support more advanced control sequences, such as conditional jumps or subroutines.

Implementation Method (Design Flow and Tools)

The Micro Control Logic is designed using **VHDL/Verilog** and developed in a modular and reusable manner. The development flow includes:

Modular Components Used: The design follows a modular approach by incorporating custom VHDL files such as MUX.vhd for selecting ALU operations and Decoder4to16.vhd for translating the operation code into individual enable signals. These components contribute to a more maintainable and extendable system architecture.

This modular approach allows easy expansion (e.g., adding multiplication or shift operations in the future).

Verification and Validation (V&V)

Below test plan is in place to validate functionality and robustness of the ALU:

Test Plan:

- **Unit Testing:** Verify each individual module such as ADD, SUB, TSA, MUX logic using directed test vectors.
- **Integration Testing:** Test interactions between register file, operand selection, and ALU blocks.
- **Timing Verification:** Confirm that control signals and outputs meet timing constraints during synthesis and simulation.
- **Edge Case Testing:** Apply edge cases such as overflow and zero-result conditions to verify flag behavior.

All tests are driven by a structured test bench, but VHDL code is excluded from this document.

Benefits, Assumptions, Risks and Issues

Benefits:

- **Customization:** Tailored to specific use-cases and extendable for future needs.
- **Educational Value:** Full control over data routing and operand selection.
- **Optimization:** Scalable for future microinstruction or CAR-based expansion

Assumptions:

- The system remains within the complexity bounds (i.e., limited number of registers and 8-bit data width).
- Future CAR integration will align with current architecture

Risks and Challenges:

- Lack of CAR limits current support for control sequences or loops.
- Debugging operand routing or register write-back logic could become complex as features scale
- Adding microcode-level control may require significant refactoring later

Conclusion

Designing a custom Microcode Control Logic unit, even without full microinstruction sequencing, offers the necessary flexibility and control to manage ALU operations reliably. This approach also lays the groundwork for integrating more advanced control mechanisms, such as a CAR and ROM-based microinstruction system, in future iterations.

References

1. Mano, M. M., & Ciletti, M. D. (2017). *Digital Design with an Introduction to the Verilog HDL, VHDL, and SystemVerilog* (6th ed.). Pearson.
→ For foundational digital logic and VHDL-based system design.
2. Pedroni, V. A. (2004). *Circuit Design with VHDL*. MIT Press.
→ Useful for writing modular and structured VHDL components like MUX and decoders.
3. Wakerly, J. F. (2005). *Digital Design: Principles and Practices* (4th ed.). Pearson.
→ For microcoded control logic and FSM-based system integration.
4. Xilinx Inc. (2020). *Vivado Design Suite User Guide: Designing with IP*. UG896 (v2020.1).
→ Referenced for modular IP design strategies even though Quartus is used; concepts still apply.
5. Intel Corporation. (2022). *Quartus Prime Pro Edition Handbook Volume 1: Design and Compilation*.
→ For synthesis flow, project structuring, and modular VHDL design in Quartus.
6. IEEE Standards Association. (2009). *IEEE Standard VHDL Language Reference Manual (IEEE Std 1076-2008)*.
→ Official VHDL syntax and behavioral modeling standards.